

Task Questions

1. Question 1

My Lambda function can access other AWS services and resources with the help of a unique role in AWS called an execution role for Lambdas. It outlines the permitted uses of my Lambda function, such as reading from and writing to S3 buckets and DynamoDB tables.

2. Question 2

In this lab, I configured an event source to cause the Lambda function to change DynamoDB. I specifically utilized Amazon S3 as the source for the event. The Lambda function was triggered by a new file uploaded to the S3 bucket, and it subsequently changed the DynamoDB database according to the new file.

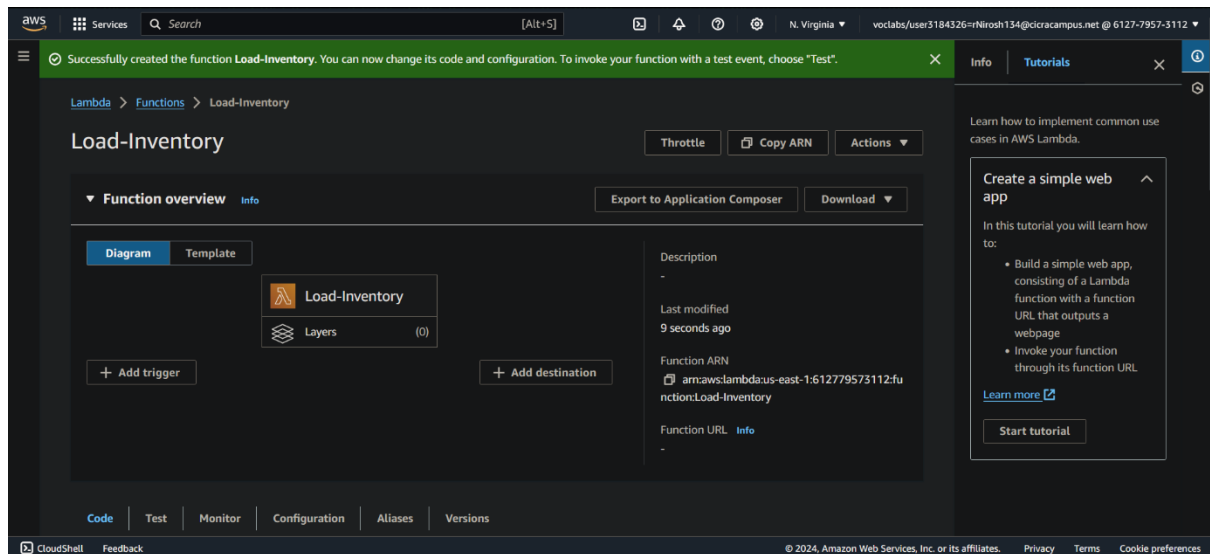
3. Question 3

The Python AWS SDK is called Boto3. I can use Python code to communicate with AWS services such as DynamoDB and S3. In this experiment, I read data from the S3 bucket using boto3 in my Lambda function, and then I updated the DynamoDB table.

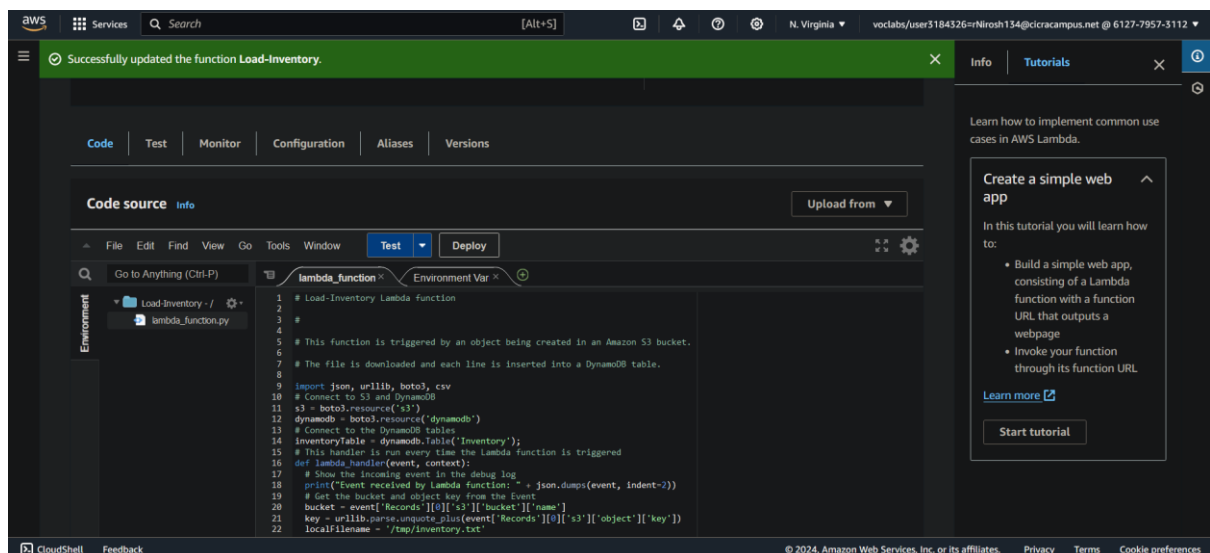
Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda

Task 1: Creating a Lambda function to load data

In the 1st task, I'm building a special tool called a Lambda function. This function is designed for a specific job which is handling inventory files. I'll grab the file from a storage area named S3 and meticulously read each line, one by one.

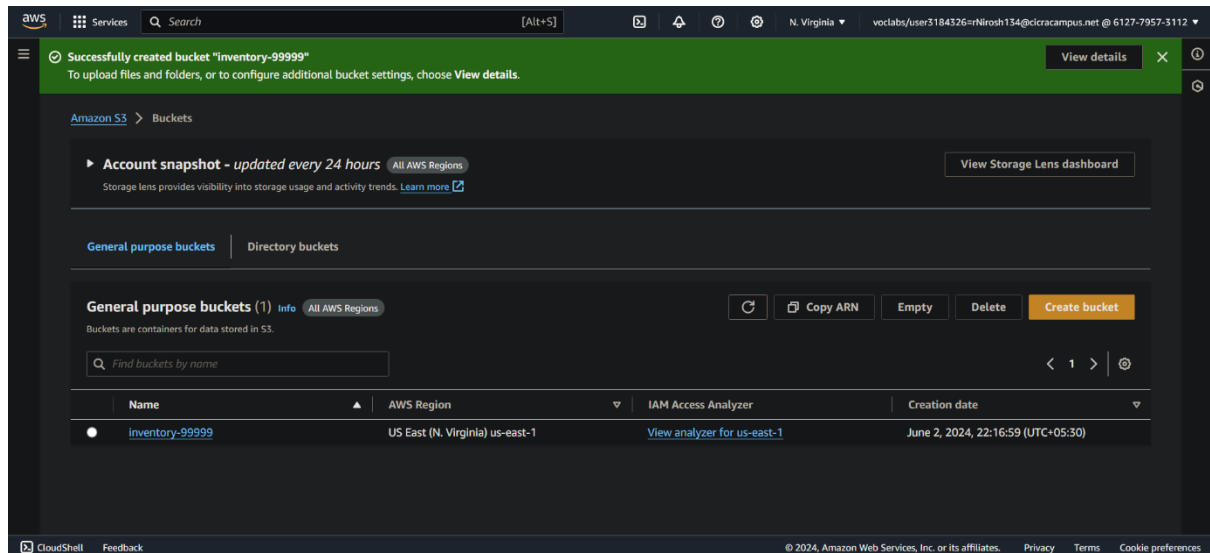


For each line, I'll carefully extract the store name, item name, and quantity. Finally, I'll store this information neatly in a database called DynamoDB.

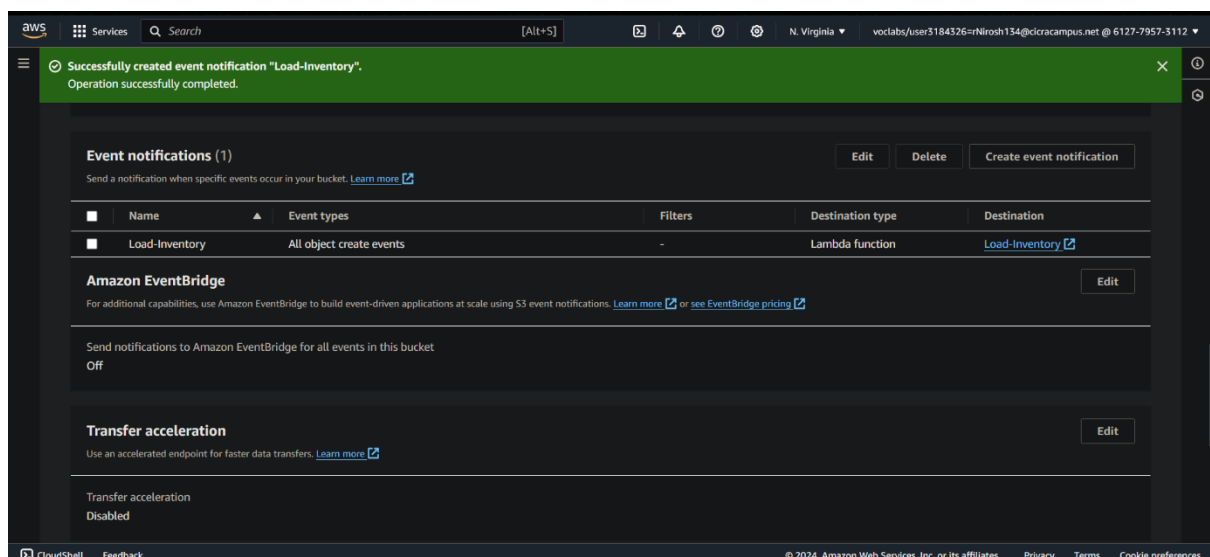


Task 2: Configuring an Amazon S3 event

In this 2nd task I'm setting up an automatic system to handle these inventory files. Stores around the world will provide them, but instead of them uploading them manually, I'm creating a system that does it automatically.

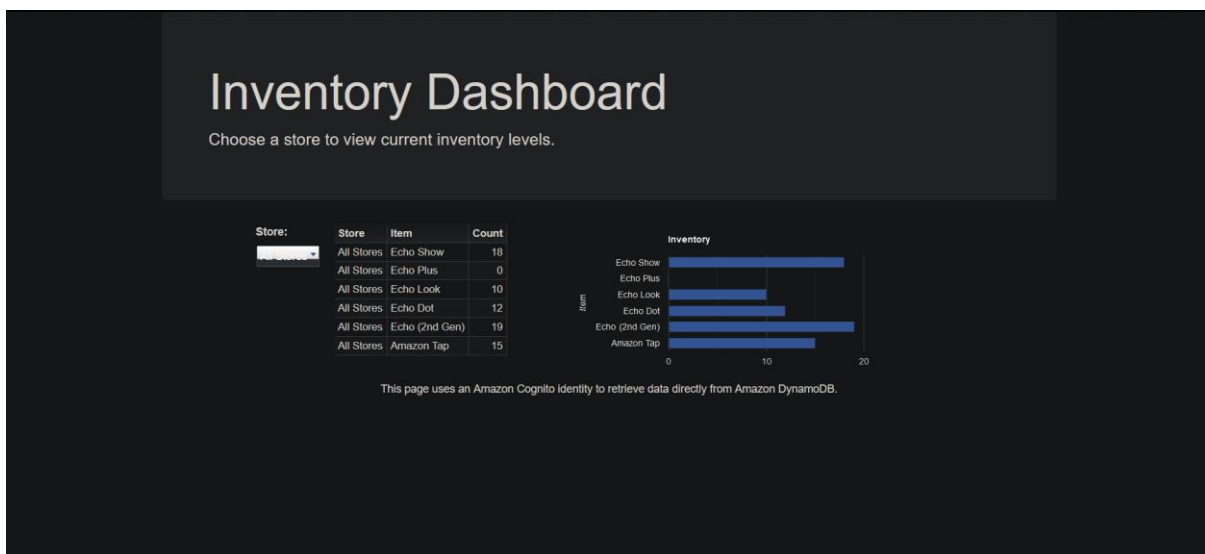
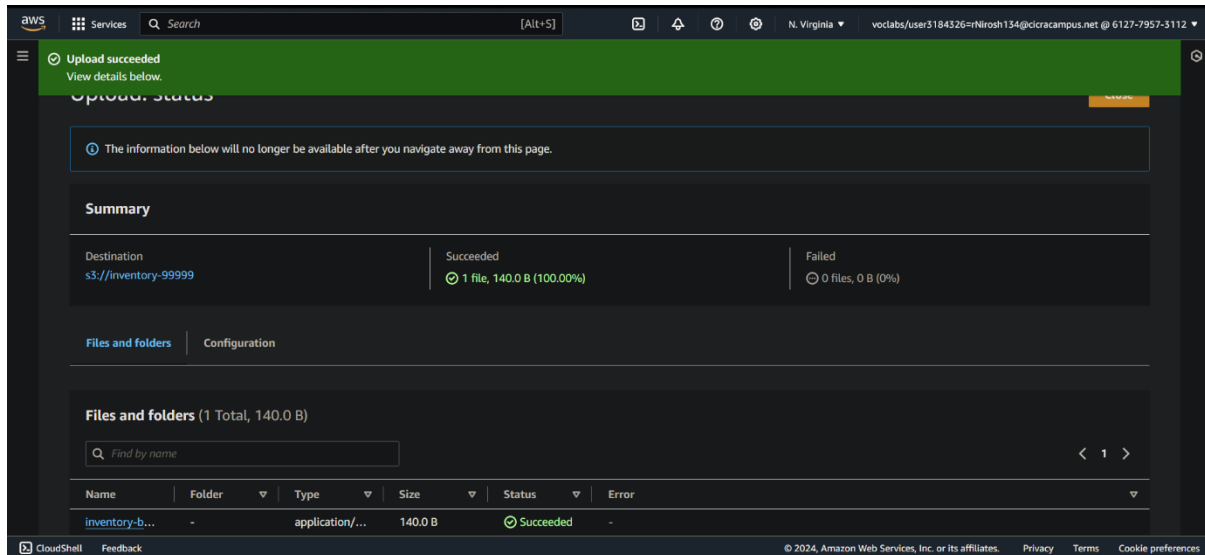


Stores will upload their files to a designated area in S3, like a filing cabinet in the cloud. Whenever a new file arrives in this area, I'll be instantly triggered! I'll then use my Lambda function (the data loader from Task 1) to process the new inventory file and store everything in DynamoDB.

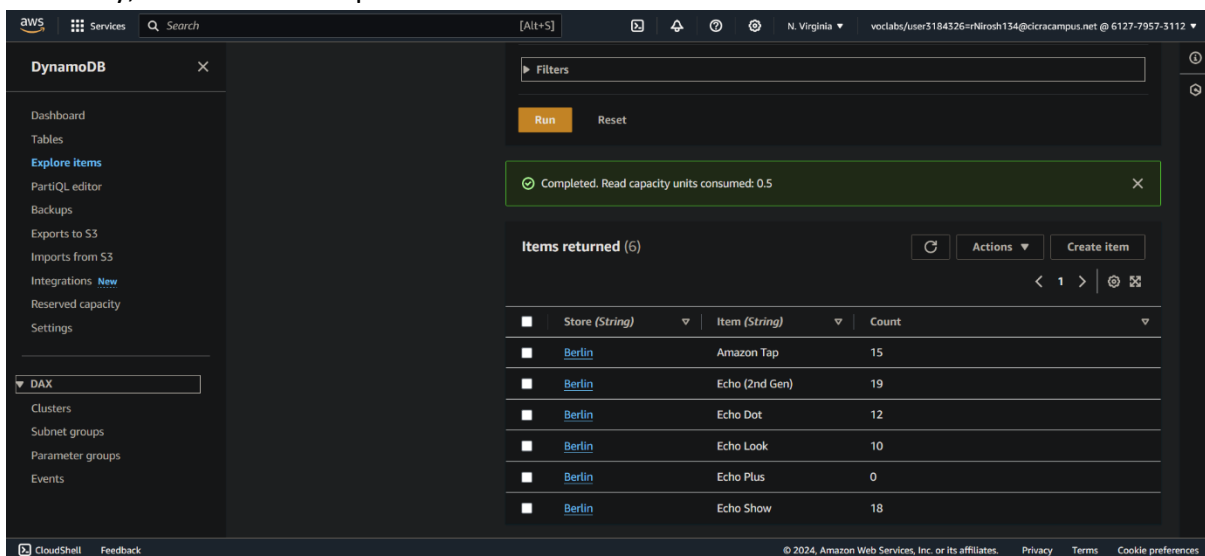


Task 3: Testing the loading process

Now that everything is set up, it's time to test it out! I'll download some sample inventory files from different stores and upload them to the S3 bucket. Once uploaded, my Lambda function will take over, automatically processing the files and storing the data in DynamoDB.

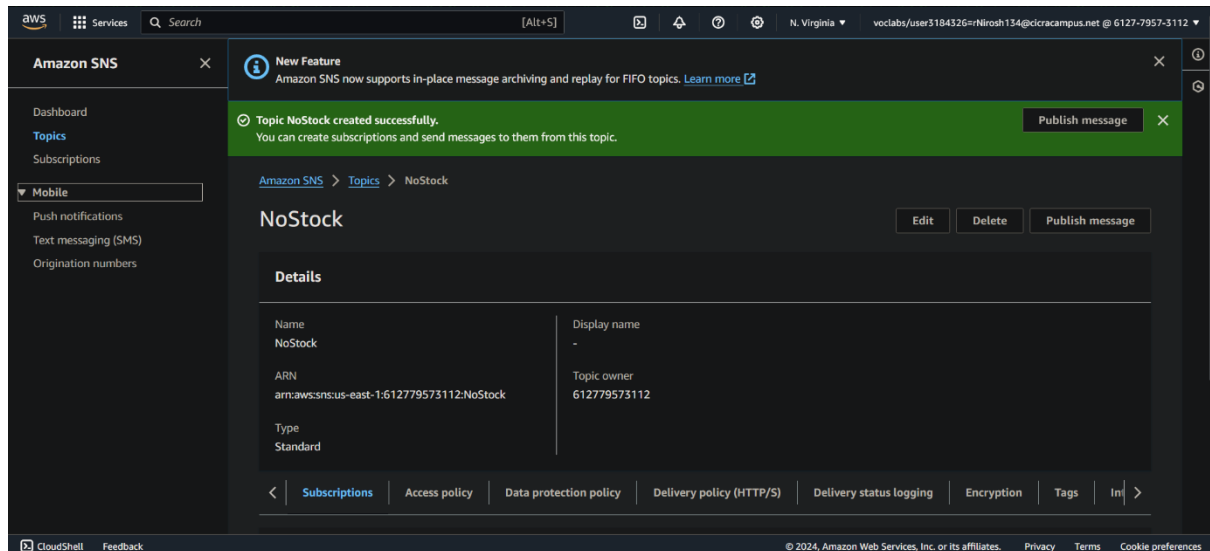


To confirm everything works, I'll check a special dashboard that displays the inventory data. This way, I can see if the uploads were successful.

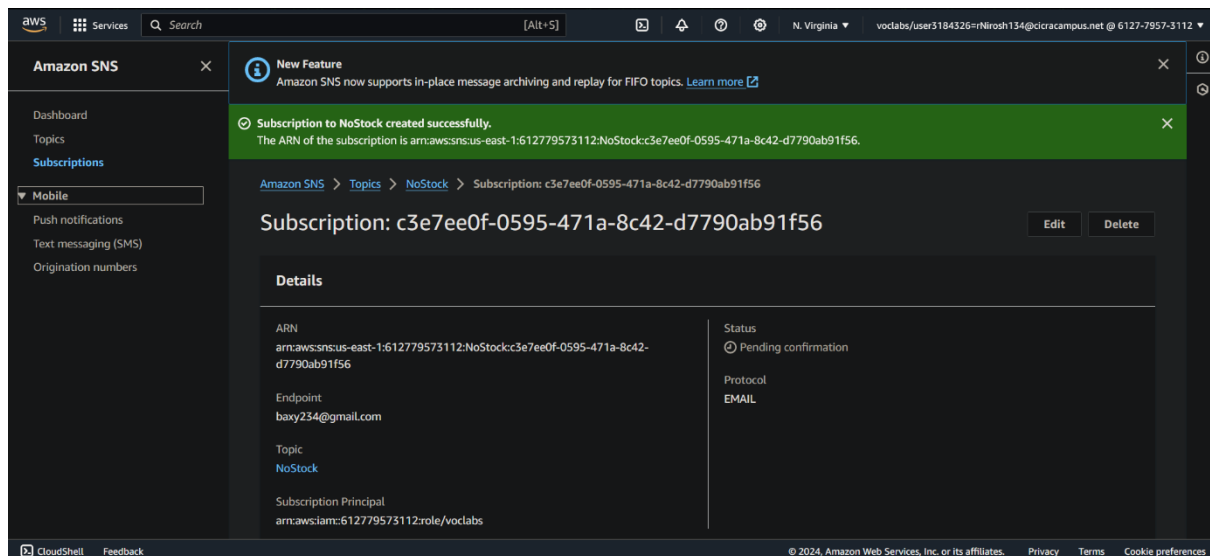


Task 4: Configuring notifications

I want to be alerted if any store runs out of stock for a particular item. To achieve this, I'll use a service called Amazon SNS, which acts like a notification center. I'll create a specific topic within SNS called "NoStock".

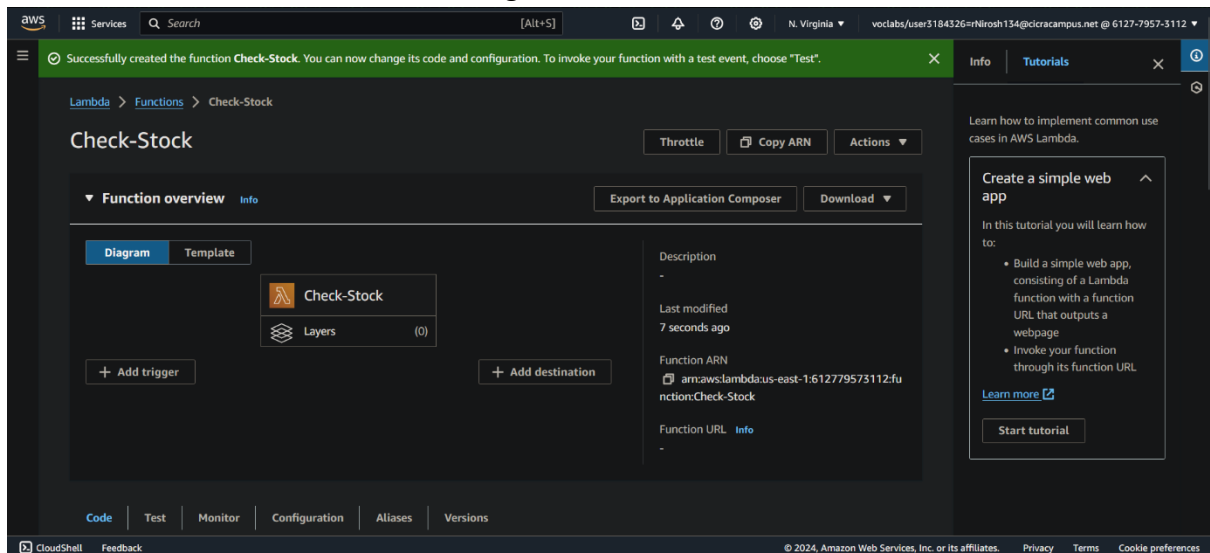


I'll subscribe my email address to this topic, so I'll receive notifications whenever something is out of stock.

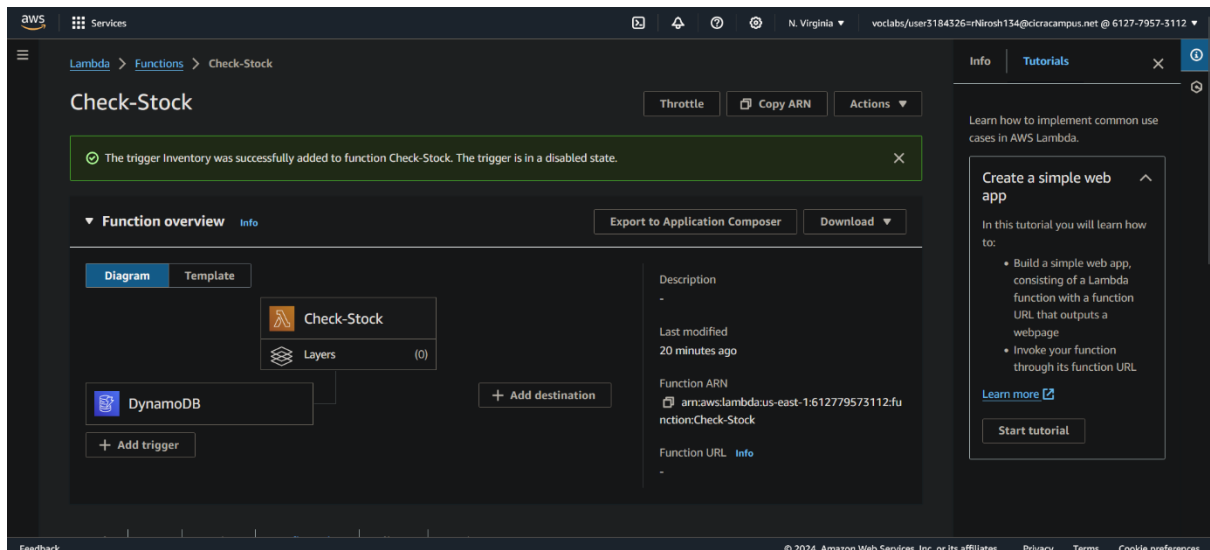
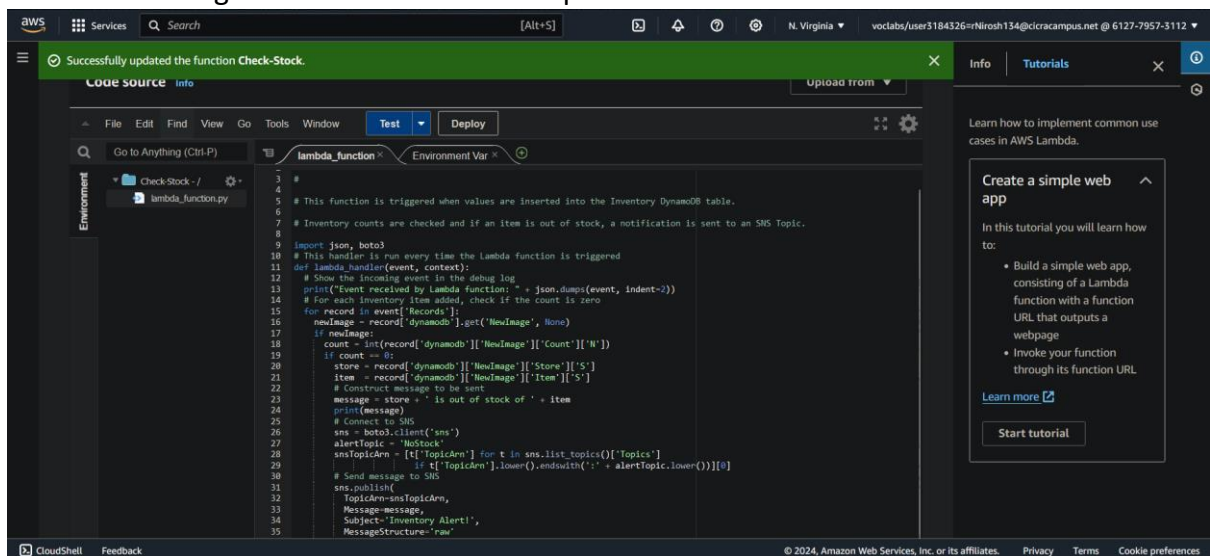


Task 5: Creating a Lambda function to send notifications

I'm creating a new Lambda function specifically designed to monitor stock levels. This function will be triggered whenever new data is added to the inventory table in DynamoDB. It will then check the quantity for each item. If an item's quantity is zero, meaning it's out of stock, the function will create a message.

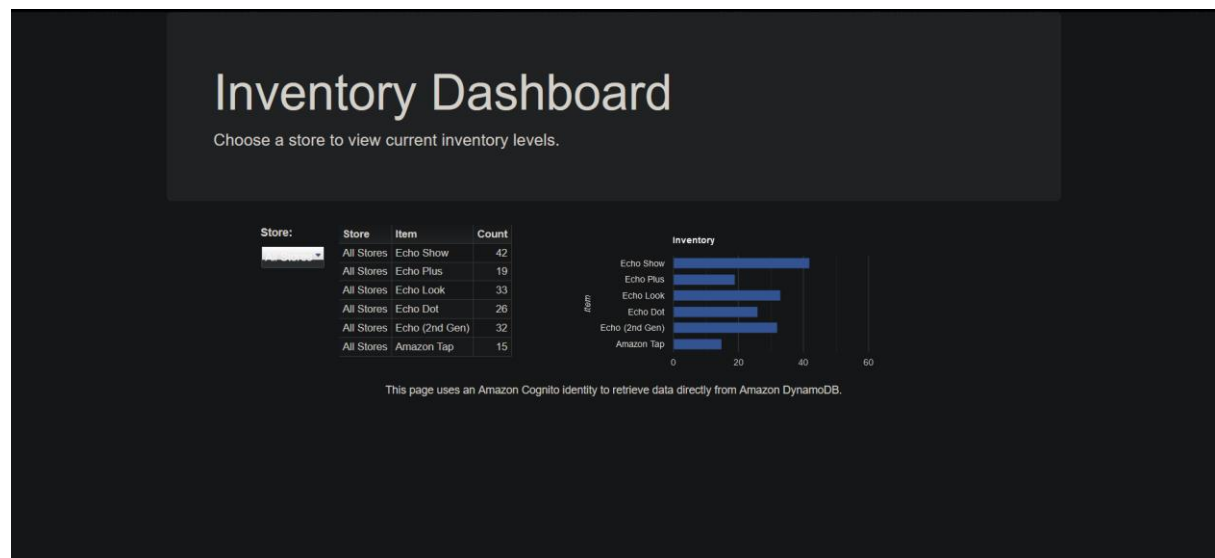
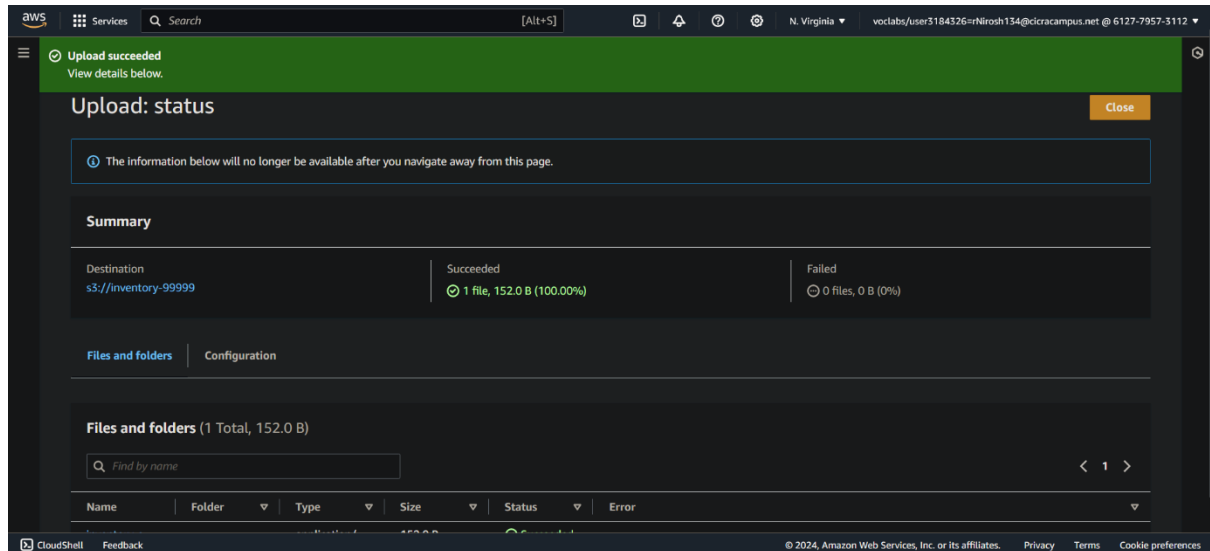


This message will include the store name and the out-of-stock item. Finally, the function will send this message to the "NoStock" SNS topic I created earlier.

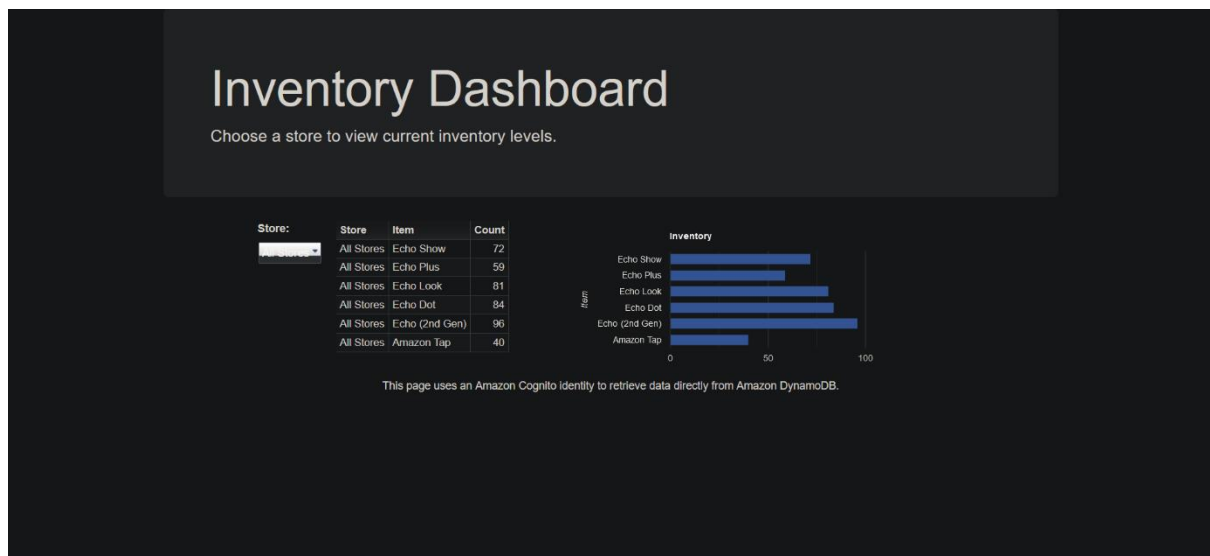
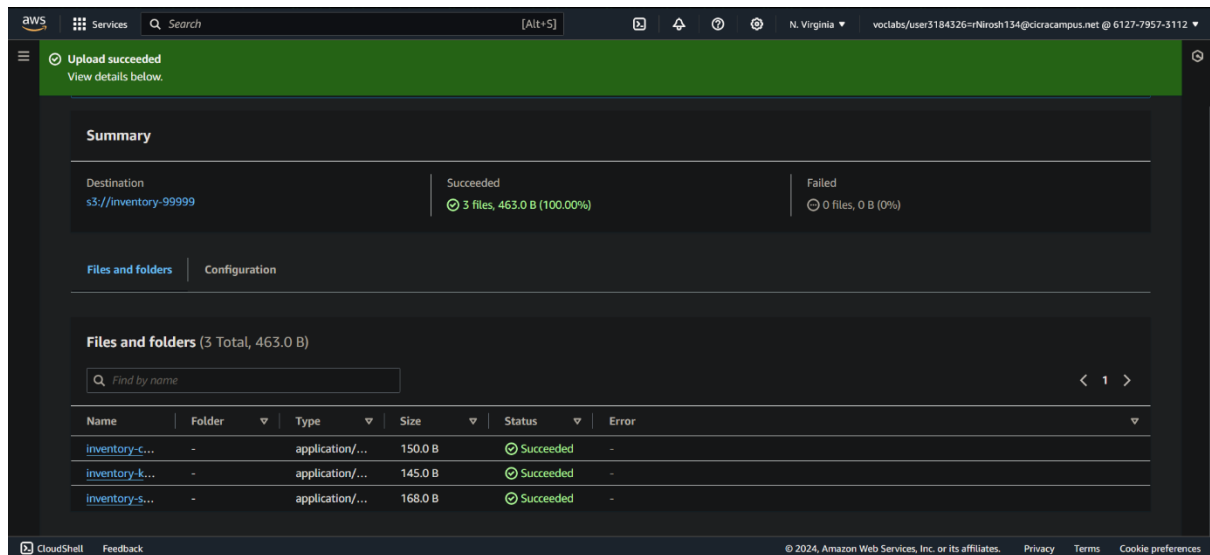


Task 6: Testing the System

Now that everything is in place, it's time for a final test! I'll upload a new inventory file to the S3 bucket. This will trigger my data loader function, which will process the file and store the data in DynamoDB. Since new data is added, my stock checker function will also be triggered.



The stock checker will then analyze the new data and identify any out-of-stock items. If there are any out-of-stock items, the checker function will send a notification to my email via the SNS topic. Uploading multiple files at once will help us see if the system can handle the additional workload and trigger the functions accordingly.



SIT233 – Cloud Computing
Module 13 Guided Lab - Implementing a Serverless
Architecture with AWS Lambda

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

Grade:

My Grade is 40/40

The screenshot shows the AWS Academy guided lab interface. The top navigation bar includes the AWS Academy logo, a hamburger menu, and the path: ACav2EN... > Assignments > Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda. The main header displays the lab title: 'Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda'. Below the header, a status bar shows 'Due No Due Date', 'Points 100', and 'Submitting an external tool'. The main content area is divided into three sections. The left section, titled 'Submitting your work', contains instructions 52 and 53, and a tip. The middle section is a large dark area with a 'Launch Terminal' button. The right section shows the 'Total score 40/40' and a list of tasks: '[Task 1A] Load-Inventory Function Crea', '[Task 1B] Load-Inventory Execution Rol', '[Task 2A] Bucket Created', '[Task 2B] Bucket Event', and '[Task 3] Database Table Data'. The bottom of the interface features a navigation bar with icons for Account, Dashboard, Courses, Calendar, Inbox, History, and Help.

aws academy

ACav2EN... > Assignments > Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda

Module 13 Guided Lab - Implementing a Serverless Architecture with AWS Lambda

Due No Due Date Points 100 Submitting an external tool

Submit Details AWS Start Lab End Lab 1:54 Instructions Grades Actions

Files README Terminal Source

EN_US

Submitting your work

52. At the top of these instructions, choose **Submit** to record your progress and when prompted, choose **Yes**.

53. If the results don't display after a couple of minutes, return to the top of these instructions and choose **Grades**.

Tip: You can submit your work multiple times. After you change your work, choose **Submit** again. Your last submission is what will be recorded for this lab.

Launch Terminal

Total score 40/40

- [Task 1A] Load-Inventory Function Crea
- [Task 1B] Load-Inventory Execution Rol
- [Task 2A] Bucket Created
- [Task 2B] Bucket Event
- [Task 3] Database Table Data

Account Dashboard Courses Calendar Inbox History Help